

Homework 02: LP reformulation, indexed models, and blending

ISyE 524 — Fall 2026

Due: Monday, September 21, 2026, at 11:59 p.m. (Madison time).

Coverage: The latter part of Lecture 3, Lecture 4, and introductory blending from Lecture 5.

Name: Dustin Pick.

Complete all three problems. Write each mathematical formulation before implementing it, define variables and units, state assumptions and variable domains, and explain your results. Problem 1 also contains written questions that need no code.

Use the course Julia environment and Julia 1.12 kernel. Run the notebook from top to bottom and save it with results visible. Submit one PDF to Gradescope, with pages matched to questions.

AI tools may help debug code you wrote, explain Julia/JuMP syntax and error messages, or critique your work. They should not replace constructing your formulation, implementation, or written solution. You are responsible for understanding and checking everything you submit. Complete the collaboration and LLM-use statement at the end.

Getting started and written answers

Follow [Assignments and PDF Submission](#). In VS Code, use **Tasks: Run Task** to run **ISyE 524: Update course repository**, then **ISyE 524: Start an assignment from its template**, entering hw02. Open `student-work/hw02/hw02.ipynb`. The copy includes its `data/` folder. An existing working copy will not be overwritten.

For **every written part**, type LaTeX mathematics in Markdown or insert a clear photograph/scan of handwritten work. Include the answer and reasoning together and label the subparts; no duplicate transcription is required. Julia/JuMP implementations belong in code cells.

Use $x_1 \geq 0$ for inline mathematics and $x_1 + x_2 \leq 5$ for display mathematics. Run the Markdown cell to render it.

For handwritten work, edit a Markdown cell in VS Code, drag in a cropped PNG/JPEG, and choose **Insert Image as Attachment**. Alternatively, put the image beside your working notebook under `student-work/hw02/images/` and insert `![My answer](images/problem1.png)`. Keep external images with the notebook. See [Images and handwritten work](#).

Back up `student-work/hw02/` separately; Git does not back up your answers or images. After completing the notebook, export it with the course PDF task, inspect `submissions/hw02.pdf` for readable mathematics, tables, outputs, and images, and submit that PDF to Gradescope.

Computational setup

Run this supplied cell first. For each LP that you solve, print its termination status and check that it is `MOI.OPTIMAL` and that a solution is available before reading variable or objective values. A solver status alone is not a mathematical explanation of the cases in Problem 1(d).

```
using JuMP, HiGHS, CSV, DataFrames, Printf
import MathOptInterface as MOI
```

Problem 1 — Equivalent LPs and solution cases

In this course, **standard form** means

$$\begin{aligned} \max_y \quad & c^\top y \\ \text{subject to} \quad & Ay \leq b, \\ & y \geq 0. \end{aligned}$$

Consider the following LP:

$$\begin{aligned} \min_x \quad & 2x_1 - x_2 + 3x_3 - 2x_4 \\ \text{subject to} \quad & 3x_2 - x_3 + 4x_4 \geq 10, \\ & 4x_1 - 7x_2 + x_3 - x_4 = 5, \\ & x_1 - 3x_3 + 8x_4 \leq 3, \\ & x_1, x_4 \geq 0, \quad x_2, x_3 \text{ unrestricted in sign.} \end{aligned}$$

1(a): Standard form and equivalence

Convert this LP to standard form. State the ordered vector of new variables and give A , b , and c explicitly. Use a different name for the new vector so it is distinguishable from the original x .

Explain how to recover every original variable from the new variables. Explain both directions of feasibility: why a feasible point of the new LP gives a feasible point of the original LP, and how any original feasible point can be represented in the new LP. State the relationship between the two objective values.

Formulation and reasoning for 1(a):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Converting Min to Max

$$\min_x \quad 2x_1 - x_2 + 3x_3 - 2x_4$$

Becomes

$$\max_x \quad -2x_1 + x_2 - 3x_3 + 2x_4$$

Converting Unrestricted Signs

x_2, x_3 unrestricted in sign

Becomes

$$u_2 \geq 0$$

$$v_2 \geq 0$$

$$x_2 = u_2 - v_2$$

$$u_3 \geq 0$$

$$v_3 \geq 0$$

$$x_3 = u_3 - v_3$$

Substituting

$$\max_x \quad -2x_1 + (u_2 - v_2) - 3(u_3 - v_3) + 2x_4$$

$$3(u_2 - v_2) - (u_3 - v_3) + 4x_4 \geq 10,$$

$$4x_1 - 7(u_2 - v_2) + (u_3 - v_3) - x_4 = 5,$$

$$x_1 - 3(u_3 - v_3) + 8x_4 \leq 3,$$

Converting Equalities to Inequalities

$$4x_1 - 7(u_2 - v_2) + (u_3 - v_3) - x_4 = 5,$$

Becomes

$$4x_1 - 7(u_2 - v_2) + (u_3 - v_3) - x_4 \leq 5$$

$$4x_1 - 7(u_2 - v_2) + (u_3 - v_3) - x_4 \geq 5$$

Converting $\geq$ to $\leq$

$$4x_1 - 7(u_2 - v_2) + (u_3 - v_3) - x_4 \geq 5$$

Becomes

$$-4x_1 + 7(u_2 - v_2) - (u_3 - v_3) + x_4 \leq -5$$

$$3(u_2 - v_2) - (u_3 - v_3) + 4x_4 \geq 10,$$

Becomes

$$-3(u_2 - v_2) + (u_3 - v_3) - 4x_4 \leq -10,$$

Rewriting in standard form (non matrix)‘

$$\begin{aligned}
& \max_x \quad -2x_1 + u_2 - v_2 - 3u_3 + 3v_3 + 2x_4 \\
& \text{subject to} \quad -3u_2 + 3v_2 + u_3 - v_3 - 4x_4 \leq -10, \\
& \quad -4x_1 + 7u_2 - 7v_2 - u_3 + v_3 + x_4 \leq -5 \\
& \quad 4x_1 - 7u_2 + 7v_2 + u_3 - v_3 - x_4 \leq 5 \\
& \quad x_1 - 3u_3 + 3v_3 + 8x_4 \leq 3, \\
& \quad x_1, x_4, u_3, v_3, u_2, v_2 \geq 0 \\
& \text{Where: } x_3 = u_3 - v_3 \\
& \quad x_2 = u_2 - v_2
\end{aligned}$$

Rewrite in Matrix Form

$$\begin{aligned}
& \max_y \quad c^\top x \\
& \text{subject to} \quad Ax \leq b, \\
& \quad x \geq 0.
\end{aligned}$$

$$A = \begin{bmatrix} 0 & -3 & 3 & 1 & -1 & -4 \\ -4 & 7 & -7 & -1 & 1 & 1 \\ 4 & -7 & 7 & 1 & -1 & -1 \\ 1 & 0 & 0 & -3 & 3 & 8 \end{bmatrix}$$

$$b = \begin{bmatrix} -10 \\ -5 \\ 5 \\ 3 \end{bmatrix}$$

$$c = \begin{bmatrix} -2 \\ 1 \\ -1 \\ -3 \\ 3 \\ 2 \end{bmatrix}$$

$$x = \begin{bmatrix} x_1 \\ u_2 \\ v_2 \\ u_3 \\ v_3 \\ x_4 \end{bmatrix}$$

Where: $x_3 = u_3 - v_3$

$x_2 = u_2 - v_2$

And: $OriginalObjective = -(NewObjective)$

x_1 and x_4 remain the same after the conversion to standard form, x_2 and x_3 require the following equations to recover their values $x_2 = u_2 - v_2$ and $x_3 = u_3 - v_3$.

The original objective is the negative of the new objective.

A feasible point of the new LP gives a feasible point of the original LP because the transformations done still result in an equivalent feasible region as well as maintain convexity. Any original feasible point can be represented in the new LP because the conversions don't lose any parts of the original set, they just translate it so it can be written in a standard form.

1(b): Solve and recover the original decisions

Implement and solve both the original LP and your standard-form LP using JuMP and HiGHS. For the standard-form implementation, use your A, b, and c rather than retyping the individual constraints.

Print both statuses, the original optimal decisions and objective value, and the original decisions and objective value recovered from the standard-form solution. Check feasibility in the **original** constraints.

When a free variable is represented as a difference of nonnegative variables, is that representation unique? Explain whether a unique original optimal solution must give a unique optimal solution in the transformed variables.

```
# Implement the original LP and the standard-form LP here.
# Recover and compare the original decisions and objective.

#Solve Original LP
using JuMP, HiGHS
Original=Model(HiGHS.Optimizer)
@variable(Original,x_1>=0)
@variable(Original,x_2)
@variable(Original,x_3)
@variable(Original,x_4>=0)
@constraint(Original,3*x_2-x_3+4*x_4>=10)
@constraint(Original,4*x_1-7*x_2+x_3-x_4==5)
@constraint(Original,x_1-3*x_3+8*x_4<=3)
@objective(Original,Min,2*x_1-x_2+3*x_3-2*x_4)

optimize!(Original)
status = termination_status(Original)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(Original) || error("No feasible optimal solution is available")

#Print Status, Original Optimal Decisions, Objective Value
println("Termination status: ", status)
println("x_1, x_2, x_3, x_4:")
println(value(x_1),",", " ",value(x_2),",", " ",value(x_3),",", " ",value(x_4))
println("Objective Value:", objective_value(Original))
```

```
println("Check Original Constraints")
#Check Feasibility of Original Constraints
println(3*value(x_2)-value(x_3)+4*value(x_4))
println(4*value(x_1)-7*value(x_2)+value(x_3)-value(x_4))
println(value(x_1)-3*value(x_3)+8*value(x_4))
```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details

Using BLAS: blastrampoline

LP has 3 rows; 4 cols; 10 nonzeros

Coefficient ranges:

Matrix [1e+00, 8e+00]

Cost [1e+00, 3e+00]

Bound [0e+00, 0e+00]

RHS [3e+00, 1e+01]

Presolving model

3 rows, 4 cols, 10 nonzeros 0s

2 rows, 3 cols, 6 nonzeros 0s

1 rows, 2 cols, 2 nonzeros 0s

0 rows, 0 cols, 0 nonzeros 0s

Presolve reductions: rows 0(-3); columns 0(-4); nonzeros 0(-10) - Reduced to empty

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Objective value : 1.59375000000e+01

P-D objective error : 1.0806733624e-16

HiGHS run time : 0.00

Termination status: OPTIMAL

x_1, x_2, x_3, x_4:

7.5937500000000001, 3.84375, 1.5312500000000007, 0.0

Objective Value:15.937500000000004

Check Original Constraints

10.0

5.0000000000000004

2.9999999999999999

Results and reasoning for 1(b): Final results are $x_1 = 7.59375$ $x_2 = 3.84375$ $x_3 = 1.53125$ $x_4 = 0$. This will result in a objective function equal to 15.9375. The constraints are all met with the solution yielding values of 10, 5 and 3 for the first, second and third constraint which had limits of ≥ 10 , $= 5$ and ≤ 3 respectively. Both x_1 and x_4 are positive as well.

```
using JuMP, HiGHS, NamedArrays
standard=Model(HiGHS.Optimizer)
#constraints are rows X's are columns
```

```

#Solve Standard Form LP
A= [
    0 -3 3 1 -1 -4
    -4 7 -7 -1 1 1
    4 -7 7 1 -1 -1
    1 0 0 -3 3 8
]

b=[
    -10
    -5
    5
    3
]

c=[
    -2
    1
    -1
    -3
    3
    2
]

x_n=[:x_1,:u_2,:v_2,:u_3,:v_3,:x_4]
constraints=[:c_1,:c_2,:c_3,:c_4]

A_n=NamedArray(A, (constraints,x_n), ("Rows", "Columns"))
b_n=NamedArray(b, (constraints,), ("Rows",))
c_n=NamedArray(c, (x_n,), ("Rows",))

@variable(standard,x_named[x_n] >=0)
@constraint(standard,constraint[i in constraints],sum(A_n[i,j]*x_named[j] for j
@objective(standard, Max, sum(c_n[j]*x_named[j] for j in x_n))

optimize!(standard)
status = termination_status(standard)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(standard) || error("No feasible optimal solution is avail

x_2=value(x_named[x_n[2]])-value(x_named[x_n[3]])
x_3=value(x_named[x_n[4]])-value(x_named[x_n[5]])

#Print Status, Original Optimal Decisions, Objective Value
println("Termination status: ", status)
println("x_1, x_2, x_3, x_4:", value(x_named[x_n[1]]),", ",x_2,", ",x_3,", ",val
println("Objective Value:", objective_value(standard))

```

```
#Check Feasibility of Original Constraints
println(3*x_2-x_3+4*value(x_4))
println(4*value(x_1)-7*x_2+x_3-value(x_4))
println(value(x_1)-3*x_3+8*value(x_4))
```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details
Using BLAS: blastrampoline
LP has 4 rows; 6 cols; 21 nonzeros

Coefficient ranges:

```
Matrix [1e+00, 8e+00]
Cost    [1e+00, 3e+00]
Bound   [0e+00, 0e+00]
RHS     [3e+00, 1e+01]
```

Presolving model

4 rows, 6 cols, 21 nonzeros 0s

3 rows, 4 cols, 10 nonzeros 0s

3 rows, 4 cols, 10 nonzeros 0s

Presolve reductions: rows 3(-1); columns 4(-2); nonzeros 10(-11)

Solving the presolved LP

Using dual simplex solver

```
Iteration      Objective      Infeasibilities num(sum)
      0      -1.7504999984e+03 Ph1: 3(5002.25); Du: 3(2.25) 0.0s
      3       1.5937500000e+01 Pr: 0(0) 0.0s
```

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 3

Objective value : -1.5937500000e+01

P-D objective error : 1.6210100436e-16

HiGHS run time : 0.00

Termination status: OPTIMAL

x_1, x_2, x_3, x_4: 7.5937500000000002, 3.84375, 1.5312500000000007, 0.0

Objective Value: -15.937500000000005

10.0

5.0000000000000004

2.9999999999999999

Results and reasoning for 1(b):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Final results are $x_1 = 7.59375$ $x_2 = 3.84375$ $x_3 = 1.53125$ $x_4 = 0$ which are the same as the original model. This will result in a objective function equal to 15.9375 but before the conversion from

standard form back to the original model it was -15.9375. The constraints are all met with the solution yielding values of 10, 5 and 3 for the first, second and third constraint which had limits of ≥ 10 , $= 5$ and ≤ 3 respectively. Both x_1 and x_4 are positive as well.

When a free variable is represented as a difference of nonnegative variables, there are many possible solutions that could fulfill the difference function but since the other constraints result in a singular optimal solution for the free variable it does not matter that the transformed variables don't have unique solutions. The transformed variables will be forced to a unique solution by the other constraints because the set must remain convex and bounded when performing these transformations.

1(c): Another transformation — bounds and a constant

Independently of parts (a)–(b), convert this LP to the same standard form:

$$\begin{aligned} \min_{p,q} \quad & 7 + 2p - q \\ \text{subject to} \quad & p + q \geq -4, \\ & p \leq 0, \quad -2 \leq q \leq 3. \end{aligned}$$

Use **two nonnegative variables**, give the transformed objective and constraints, and explain how to recover p , q , and the original objective value. Do not introduce a difference of two variables when a sign change or shift suffices. No implementation or numerical solution is required.

Formulation and reasoning for 1(c):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Converting min to max

$$\begin{aligned} \min_{p,q} \quad & 7 + 2p - q \\ & \text{Becomes} \\ \max_{p,q} \quad & -7 - 2p + q \end{aligned}$$

Removing constant from objective

$$\begin{aligned} & 7 + 2p - q \\ & \text{Becomes} \\ & 2p - q' \\ \text{Where: } & q' = q + 7 \end{aligned}$$

Converting bounded to non-negative

$$\begin{aligned} & -2 \leq q' - 7 \leq 3 \\ & \text{Becomes} \\ & 0 \leq q'' \\ & q'' \leq 5 \end{aligned}$$

Reversing Inequalities

$$p + q'' - 7 \geq -4$$

Becomes

$$-p - q'' \leq -3$$

$$p \leq 0$$

Becomes

$$-p \geq 0$$

Rewrite In standard form (non-matrix)

$$\begin{aligned} \max_{p,q} \quad & -2p + q'' \\ \text{subject to} \quad & -p - q'' \leq -3 \\ & q'' \leq 5 \\ & -p \geq 0 \\ & q'' \geq 0 \end{aligned}$$

Where: $q'' = q' + 2$
 $q' = q + 7$

Rewrite in Matrix Form

$$\begin{aligned} \max_y \quad & c^\top x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

$$A = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

$$b = \begin{bmatrix} -3 \\ 5 \end{bmatrix}$$

$$c = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

$$x = \begin{bmatrix} -p \\ q'' \end{bmatrix}$$

Where: $q'' = q + 9$

$$p = p$$

And: $OriginalObjective = -(NewObjective) + 9$

1(d): Four solution cases

For each LP below, decide whether it is **infeasible**, **unbounded in objective**, has a **unique optimum**, or has **multiple optima**. Justify each conclusion mathematically. A sketch is welcome; no code is required.

For multiple optima, give the **entire set** of optimal solutions and the optimal value. For infeasibility, derive a contradiction from the constraints. For an unbounded objective, exhibit feasible points depending on a parameter whose objective improves without limit. For a unique optimum, give the point and value and explain uniqueness.

Case A

$$\begin{array}{ll}\max & 4x_1 + x_2 \\ \text{subject to} & 8x_1 + 2x_2 \leq 16, \\ & 5x_1 + 2x_2 \leq 12, \\ & x_1, x_2 \geq 0.\end{array}$$

Case B

$$\begin{array}{ll}\max & x_1 + x_2 \\ \text{subject to} & 2x_1 + 2x_2 \leq 8, \\ & x_1 - x_2 \geq 5, \\ & x_1, x_2 \geq 0.\end{array}$$

Case C

$$\begin{array}{ll}\max & -x_1 + 3x_2 \\ \text{subject to} & x_1 - x_2 \leq 4, \\ & x_1 + 2x_2 \geq 4, \\ & x_1, x_2 \geq 0.\end{array}$$

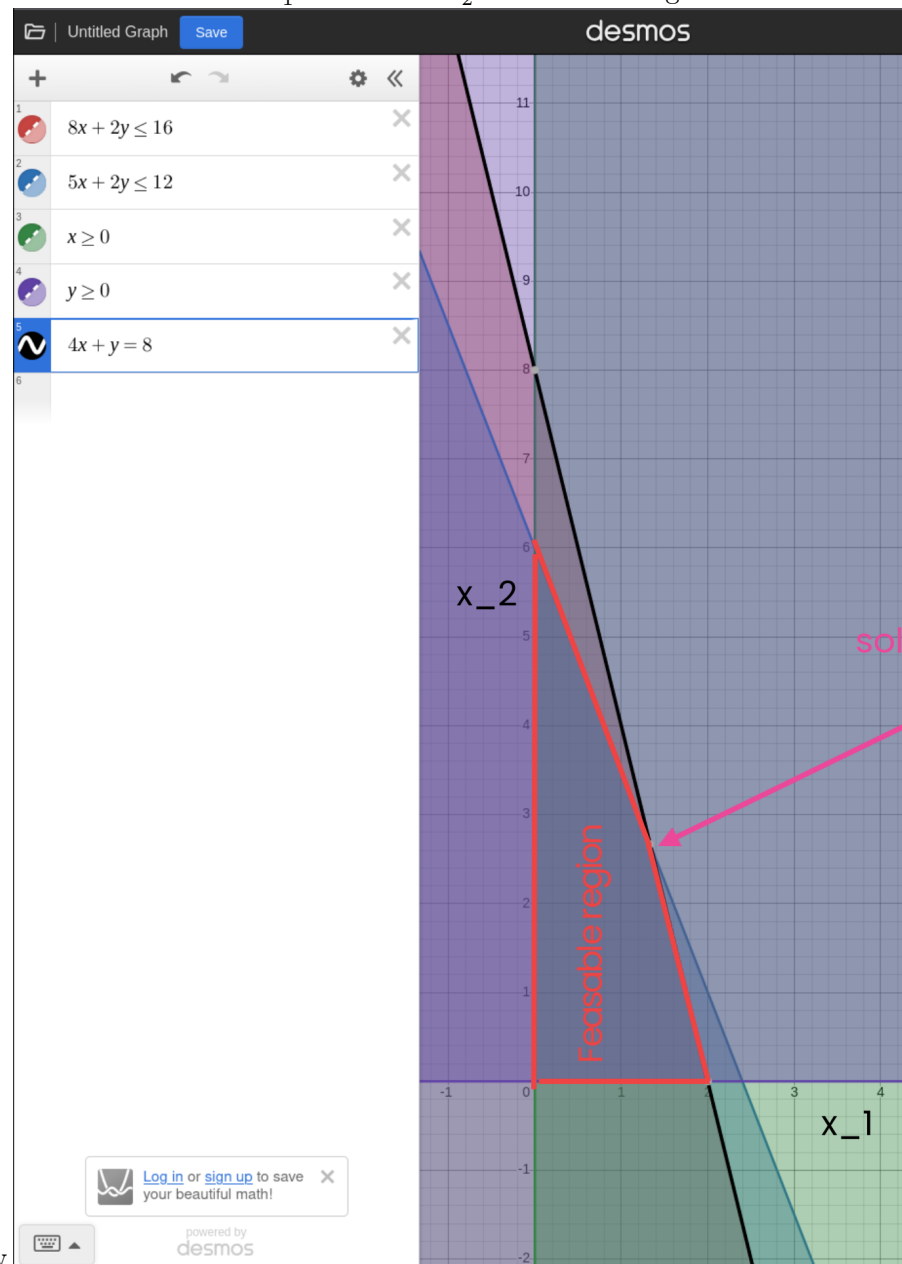
Case D: Change **max** to **min** in Case C, leaving the objective expression and all constraints unchanged.

Compare Cases C and D. Is their common feasible region bounded? What does this example show about the distinction between an unbounded feasible region and an unbounded optimization problem?

Classifications and reasoning for 1(d):

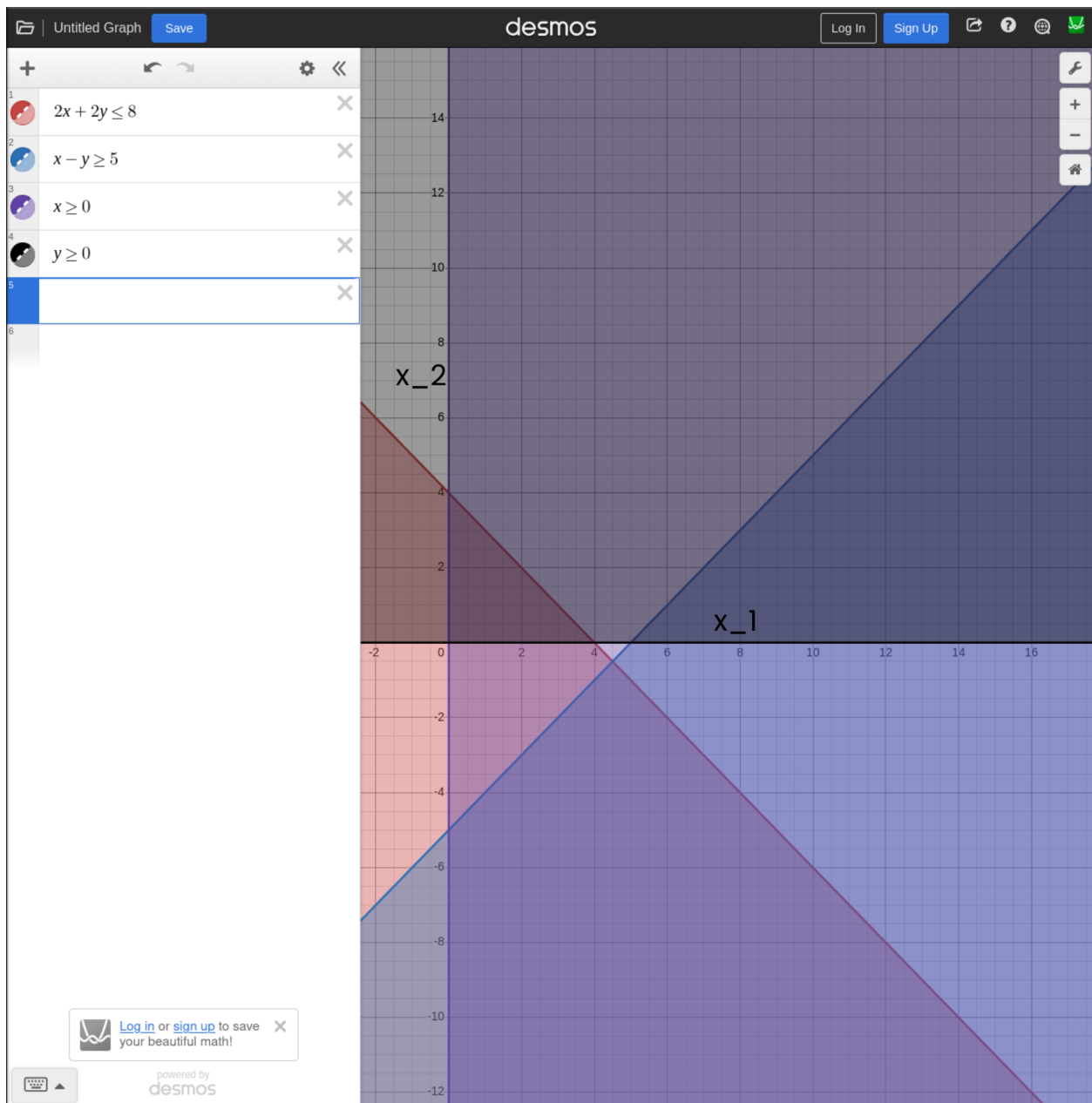
Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Case A This LP has a unique optimum solution which occurs at $x_1 = 1.33$ and $x_2 = 2.66$ resulting in

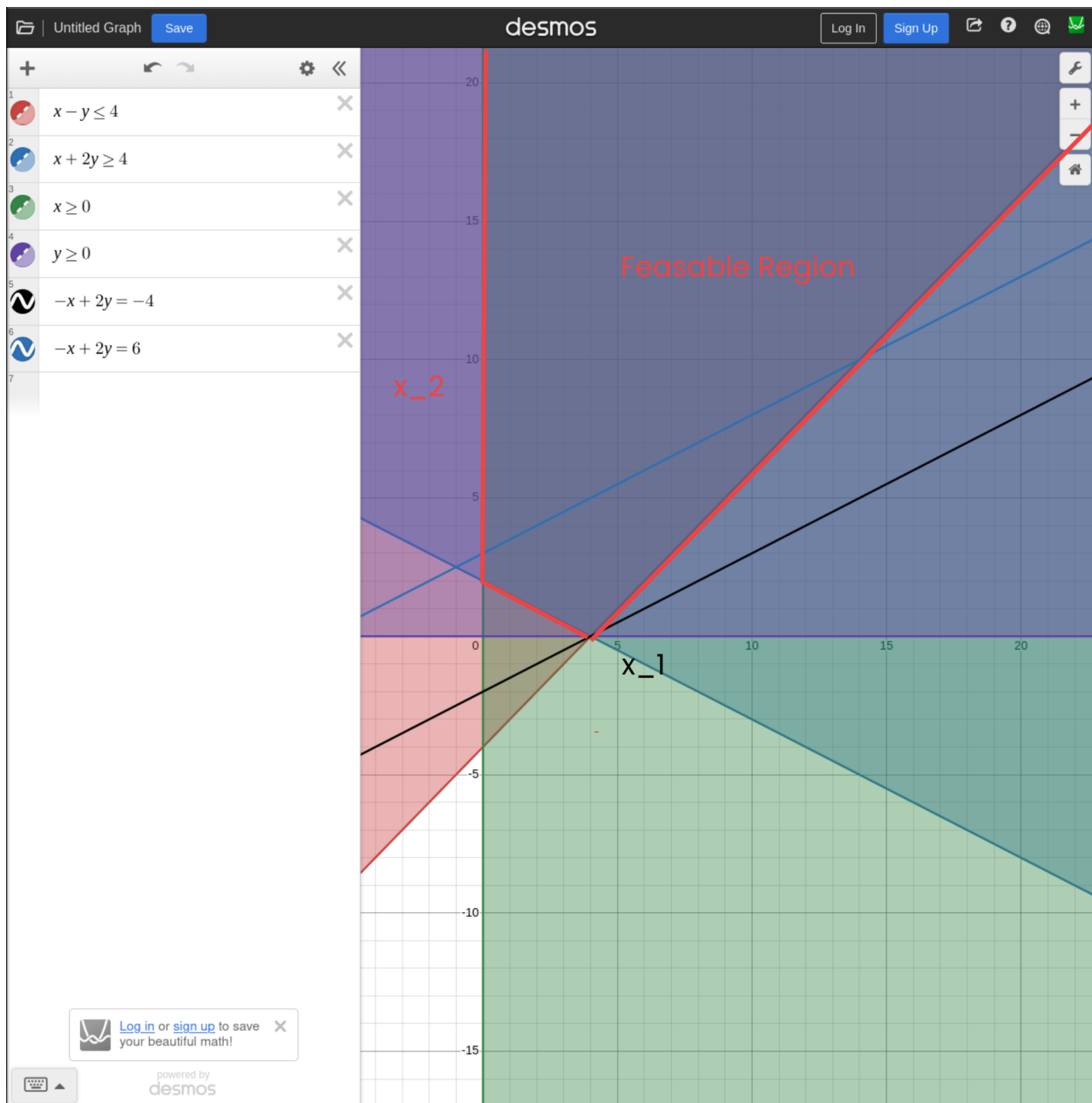


a objective function equal to 8. Graph Below

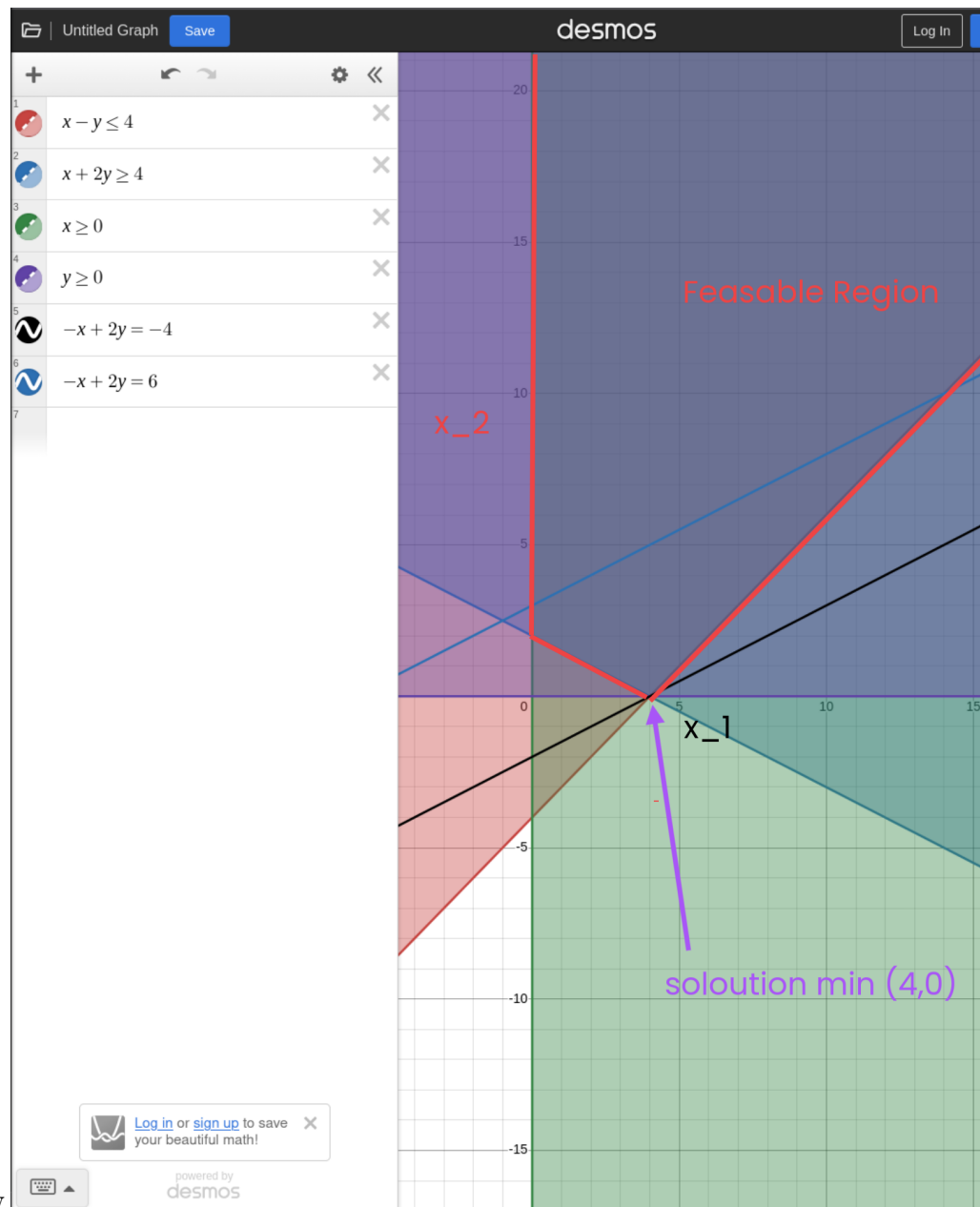
Case B This LP is infeasible because its constraints don't produce a convex set. This is because you cannot select a point in $2x_1 + 2x_2 \leq 8$ and $x_1 - x_2 \geq 5$ that also satisfies $x_1 \geq 0$. Graph Below



Case C This LP is unbounded because as you move along the $x_1 - x_2 \leq 4$ constraint you never reach a another constraint in the maximum direction. Graph Below



Case D This LP has a unique solution which occurs at $x_1 = 4$ and $x_2 = 0$ resulting in a objective



function equal to -4. Graph Below

When comparing cases C to D you can find that the feasible region is only bounded in one direction. This shows that you don't need a bounded feasible region to have a bounded optimization problem you just need to make sure you are optimizing towards a area of the region that is bounded.

Problem 2 — Mission Impossible: a general indexed model

Jacob's Meat Alternative Plant (JMAP) prepares plant-based substitutes for several pork products. A product p in a set P can be prepared and sold using a method c in a set C .

For each product p , at most b_p units are available each day. Method c can process at most u_c units per day across **all** products. Selling one unit of product p prepared by method c brings in h_{pc} dollars **before** the processing cost q_c dollars per unit is deducted.

One unit of a product uses one unit of processing capacity and produces one unit for sale. Each unit is assigned to at most one method. Fractional units are allowed in this continuous planning model. All product–method combinations in the data are allowed. Prepared products can all be sold at the stated prices. Product may be left unused with no salvage value or disposal cost; there are no minimum sales requirements. Other costs are fixed and do not affect this decision.

2(a): General mathematical formulation

Write an LP that maximizes JMAP’s daily net profit. Define the decisions and their units. Use $P, C, b_p, u_c, h_{pc}, q_c$ to write a **general indexed formulation** that works for any numbers of products and methods. Explain the meaning and direction of each family of constraints. Do not substitute the numerical data into this written model.

Formulation and reasoning for 2(a):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Sets: P (products), C (Method of Sale)

Parameters: b_p (units available per product), u_c (units available per process), h_{pc} (sale gross profit), q_c (sale costs per process)

Objective function sums net profit of every product across each process and maximizes it.

$$\max \sum_{p \in P} \sum_{c \in C} (h_{pc} - q_c) x_{pc}$$

The first set of constraints sums the number of products per process and makes sure it is less than or equal to the limit of units that can be processed by that process.

$$\sum_{p \in P} x_{pc} \leq u_c \quad \forall c \in C$$

The second set of constraints sums the number of products processed and makes sure it is less than or equal to the limit of units available to sell that day.

$$\sum_{c \in C} x_{pc} \leq b_p \quad \forall p \in P$$

The final constraint makes sure a non-negative amount of products is processed

$$x_{pc} \geq 0 \quad \forall c \in C, p \in P$$

Supplied data for Problem 2

The data/ folder contains three CSV files. Their labels connect the tables; their row order has no mathematical significance.

File	Columns	Meaning and units
products.csv	product, availability	Product label and b_p (units/day)
methods.csv	method, capacity, cost	Method label, u_c (units/day), q_c (dollars/unit)
revenues.csv	product, method, revenue	One row per pair; h_{pc} (dollars/unit)

The supplied code creates P and C, dictionaries b, u, q, and a dictionary h indexed by pairs. For example, b["Ham"] is the daily Ham availability, and h[("Ham", "Fresh")] is its revenue per unit when prepared Fresh. You do not need to write or modify the data-loading code.

The supplied loader works when the kernel starts beside this notebook or at the course repository root. From the repository root, it uses student-work/hw02/ when that copy exists; otherwise it uses the published template. Keep the copied data/ folder beside your working notebook. A missing-file message identifies the folder to check.

```
# Locate this assignment, including when a notebook kernel starts at the repo root
notebook_dirs = [
    @__DIR__,
    joinpath(pwd(), "student-work", "hw02"),
    joinpath(pwd(), "assignments", "hw02"),
]
notebook_index = findfirst(
    directory -> isfile(joinpath(directory, "hw02.ipynb")), notebook_dirs,
)
isnothing(notebook_index) && error(
    "Open the course repository or run from the folder containing hw02.ipynb."
)
data_dir = joinpath(notebook_dirs[notebook_index], "data")
for filename in ("products.csv", "methods.csv", "revenues.csv")
    isfile(joinpath(data_dir, filename)) || error(
        "Missing $(filename) in $(data_dir). Keep the complete data folder beside"
    )
end
products = CSV.read(joinpath(data_dir, "products.csv"),
    DataFrame; stringtype=String)
methods = CSV.read(joinpath(data_dir, "methods.csv"),
    DataFrame; stringtype=String)
revenues = CSV.read(joinpath(data_dir, "revenues.csv"),
    DataFrame; stringtype=String)
```

```

P = products.product
C = methods.method
b = Dict{row.product => row.availability
    for row in eachrow(products)}
u = Dict{row.method => row.capacity for row in eachrow(methods)}
q = Dict{row.method => row.cost for row in eachrow(methods)}
h = Dict{(row.product, row.method) => row.revenue
    for row in eachrow(revenues)}

# Check labels and completeness before using the data.
@assert length(unique(P)) == length(P)
@assert length(unique(C)) == length(C)
@assert nrow(revenues) == length(h) == length(P) * length(C)
@assert all(haskey(h, (p, c)) for p in P, c in C)

println("Products: ", join(P, ", "))
println("Methods: ", join(C, ", "))
println("Revenue entries: ", length(h))

```

```

Products: Ham, PorkBelly, Picnic, Bacon, Tenderloin
Methods: Fresh, SmokedRegular, SmokedOvertime, Cured, Pickled
Revenue entries: 25

```

2(b): Implement, solve, and interpret

Implement your general formulation using the supplied sets and dictionaries, indexed variables, and sums. It should continue to work if the tables contain different product or method labels or different numbers of rows.

Solve using HiGHS. Print the status and maximum daily profit. Report every positive product–method allocation, the total amount used and unused for each product, and the capacity used and unused for each method. Check that the reported plan satisfies all supply and capacity limits, and independently calculate its net profit from the allocation.

Which processing capacities are fully used, and which have spare capacity? Explain why spare capacity in one method need not eliminate the value of capacity in another.

```

# Build and solve the indexed JMAP model here.
# Report allocations, supply/capacity use, and net profit.

using JuMP, HiGHS

jmap=Model(HiGHS.Optimizer)

@variable(jmap, x[P, C] >=0)

@constraint(jmap, process_limit[c in C],

```

```

    sum(x[p,c] for p in P) <= u[c])

@constraint(jmap,product_limit[p in P],
    sum(x[p,c] for c in C) <= b[p])

@objective(jmap, Max, sum((h[p, c] - q[c]) * x[p, c] for p in P, c in C))

optimize!(jmap)
status = termination_status(jmap)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(jmap) || error("No feasible optimal solution is available")
solutions= Dict{(p,c) => value(x[p,c]) for p in P, c in C)
solution= Dict{(p,c) => value(x[p,c]) for p in P, c in C if value(x[p,c]) > 1e-6}

#Print Status, Objective Value,
println("Termination status: ", status)
println("Maximum Daily Profit: ",objective_value(jmap), " dollars")

#Print Product and amount Produced\
println("")
for p in P, c in C
    if haskey(solution, (p,c))
        println("Product: ",p," Process: ", c," Units Produced: ",solution[(p,c)])
    end
end

#Print Product Usage
println("")
for p in P
    product_use=sum(i for ((product,c),i) in solution if product == p; init=0.0)
    println("Product: ", p, " Total units used: ", product_use)
    product_left=b[p]-product_use
    println("Product: ", p, " Total units left: ", product_left)
end

#Print Capacity Usage
println("")
for c in C
    capacity_use=sum(i for ((p,capacity),i) in solution if capacity == c; init=0.0)
    println("Process: ", c, " Total Capacity used: ", capacity_use)
    capacity_left=u[c]-capacity_use
    println("Process: ", c, " Total capacity left ", capacity_left)
end

#Print Profit
println("")
profit=sum(i*(h[p, c] - q[c]) for ((p,c),i) in solution; init=0.0)

```

```
println("Profit calculated from outputs: ", profit, " dollars")
```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license

Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details

Using BLAS: blastrampoline

LP has 10 rows; 25 cols; 50 nonzeros

Coefficient ranges:

Matrix [1e+00, 1e+00]

Cost [1e+00, 1e+01]

Bound [0e+00, 0e+00]

RHS [1e+02, 5e+03]

Presolving model

10 rows, 24 cols, 48 nonzeros 0s

7 rows, 15 cols, 25 nonzeros 0s

Presolve reductions: rows 7(-3); columns 15(-10); nonzeros 25(-25)

Solving the presolved LP

Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	-1.0299901578e+02	Ph1: 7(24); Du: 14(102.999)	0.0s
10	-1.4725000000e+04	Pr: 0(0)	0.0s

Performed postsolve

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 10

Objective value : 1.4725000000e+04

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.00

Termination status: OPTIMAL

Maximum Daily Profit: 14725.0 dollars

Product: Ham Process: Fresh Units Produced: 380.0

Product: Ham Process: Cured Units Produced: 100.0

Product: PorkBelly Process: SmokedRegular Units Produced: 230.0

Product: PorkBelly Process: Pickled Units Produced: 170.0

Product: Picnic Process: Pickled Units Produced: 230.0

Product: Bacon Process: Fresh Units Produced: 310.0

Product: Bacon Process: SmokedRegular Units Produced: 190.0

Product: Tenderloin Process: SmokedOvertime Units Produced: 245.0

Product: Ham Total units used: 480.0

Product: Ham Total units left: 0.0

Product: PorkBelly Total units used: 400.0

Product: PorkBelly Total units left: 0.0

Product: Picnic Total units used: 230.0

Product: Picnic Total units left: 0.0

Product: Bacon Total units used: 500.0
Product: Bacon Total units left: 0.0
Product: Tenderloin Total units used: 245.0
Product: Tenderloin Total units left: 0.0

Process: Fresh Total Capacity used: 690.0
Process: Fresh Total capacity left 4310.0
Process: SmokedRegular Total Capacity used: 420.0
Process: SmokedRegular Total capacity left 0.0
Process: SmokedOvertime Total Capacity used: 245.0
Process: SmokedOvertime Total capacity left 5.0
Process: Cured Total Capacity used: 100.0
Process: Cured Total capacity left 0.0
Process: Pickled Total Capacity used: 400.0
Process: Pickled Total capacity left 0.0

Profit calculated from outputs: 14725.0 dollars

Results and interpretation for 2(b):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

The maximum profit from this LP is 14,725 dollars. This is achieved by producing 380 units Fresh Ham, 100 Units Cured Ham, 230 units Regular Smoked Pork Belly, 170 Units Pickled Pork Belly, 230 units Pickled Picnic, 310 units Fresh Bacon, 190 units Regular smoked bacon, and 245 units Overtime Smoked Tenderloin.

This results in all Products being processed with 4310 units of fresh capacity left, 0 regular smoke units of capacity left, 5 units of overtime smoking left, 0 units of curing and pickling capacity left.

Spare capacity in one method need not eliminate the value of capacity in another in this problem because different methods have different cost and different meats have different profits. Thus you have to consider net profit rather than gross profit which means that different products may profit better in different methods even though gross profit on one method may be greatest,

2(c): Why independent choices can fail

Suppose a manager proposes: "For each product, prepare all available units using whichever method gives the largest profit per unit."

Using the data, show that this rule cannot produce a feasible plan, even if ties between equally profitable methods are resolved favorably. Identify a method whose capacity would be exceeded and quantify the overload. Explain what coupling between decisions the general LP captures that the manager's rule misses. No additional optimization run is required.

Reasoning for 2(c):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

The method which the manager proposes will quickly overload the regular smoking method since it has the greatest profit for nearly every product. It would be overloaded by 955 units because you would make every product except for the ham smoked regular. The LP considers capacitys for each method as well as units available and thus is able to properly allocate capacity to the product which makes the most profit as well as fully utilize processes without overloading them.

Problem 3 — Alloy blending

Linderoth Optimal Steel Emporium (LOSE) must deliver **at least 500 tons** of a steel blend. The customer permits excess delivery but pays no extra for it. All material blended is delivered; its composition must satisfy these requirements.

Element	Minimum (%)	Maximum (%)
Carbon (C)	2	3
Copper (Cu)	0.4	0.6
Manganese (Mn)	1.2	1.65

Seven raw materials are available. Their elemental compositions, available quantities, and costs are:

Raw	C (%)	Cu (%)	Mn (%)	Available (tons)	Cost (dollars/ton)
F	2.5	0	1.3	400	200
U	3	0.3	1.8	300	250
E	1.5	0.5	1.5	600	150
L	0	0.9	0	500	220
O	0.4	0.9	1.1	200	240
N	0	0.4	1.2	300	200
M	0	0.6	0	250	165

Quantities are continuous. Assume no material is lost in blending, and elemental composition is the mass-weighted average of the raw materials' compositions. Only the quantities actually used incur the listed costs.

3(a): Mathematical formulation

Formulate an LP minimizing the cost of the delivery. Define variables and units; include the delivery requirement, raw-material availability limits, and both the lower and upper composition requirements for every element.

You may use indexed notation: J for raw materials, E for elements, c_j for cost, u_j for availability, γ_{ej} for the percentage of element e in raw j , α_e, β_e for minimum and maximum percentages, and $d = 500$ for the minimum delivery.

Starting with a weighted-average expression for an element's percentage, explain why you can rewrite each quality requirement as a linear constraint. Identify why multiplication by the denominator is valid here. Use a consistent convention for percentages: for example, 2 and 3, or 0.02 and 0.03, throughout.

Formulation and reasoning for 3(a):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Sets: J for raw materials, E for elements

Parameters: c_j for cost, u_j for availability, γ_{ej} for the percentage of element e in raw j , α_e, β_e for minimum and maximum percentages, and $d = 500$ for the minimum delivery

$$\min \sum_{j \in J} c_j x_j$$

$$0 \leq x_j \leq u_j \quad \forall j \in J$$

$$\sum_{j \in J} x_j \geq d \quad \forall j \in J$$

$$\sum_{j \in J} x_j \alpha_e \leq \sum_{j \in J} \gamma_{ej} x_j \leq \sum_{j \in J} x_j \beta_e \quad \forall e \in E$$

You can write the requirements for an elements percentage because a weighted average is just a linear sum. This is because a weighted average is represented as $\bar{x} = \frac{\sum_{i=1}^n W_n x_n}{\sum_{i=1}^n W_n}$ and since this is a percentage the denominator goes to 1 and the equation just becomes a linear summation of the weights as percentages. Thus the w_n in the numerator can be expressed as a minimum and maximum and set as the inequality

$$\sum_{j \in J} x_j \alpha_e \leq \sum_{j \in J} \gamma_{ej} x_j \leq \sum_{j \in J} x_j \beta_e \quad \forall e \in E$$

Supplied data for Problem 3

The following cell supplies the table values using the indexed notation above. In $\gamma[(e, j)]$, the element comes first and the raw material second. These names replace the Problem 2 data names where they overlap; finish Problem 2 before running this section.

```
J = ["F", "U", "E", "L", "O", "N", "M"]
E = ["C", "Cu", "Mn"]
c = Dict(zip(J, [200, 250, 150, 220, 240, 200, 165]))
u = Dict(zip(J, [400, 300, 600, 500, 200, 300, 250]))
α = Dict(zip(E, [2.0, 0.4, 1.2]))
β = Dict(zip(E, [3.0, 0.6, 1.65]))
d = 500.0

# Rows follow J; columns follow E. Entries are percentages.
composition = [
    2.5  0.0  1.3
    3.0  0.3  1.8
    1.5  0.5  1.5
    0.0  0.9  0.0
    0.4  0.9  1.1
    0.0  0.4  1.2
    0.0  0.6  0.0
]
γ = Dict{(e, j) => composition[k, l]}
    for (k, j) in enumerate(J), (l, e) in enumerate(E)
```

Dict{Tuple{String, String}, Float64} with 21 entries:

```
("Cu", "U") => 0.3
("Cu", "E") => 0.5
("C", "U")  => 3.0
("C", "F")  => 2.5
("C", "O")  => 0.4
("Mn", "M") => 0.0
("Mn", "O") => 1.1
("Mn", "N") => 1.2
("C", "L")  => 0.0
("Cu", "F") => 0.0
("Cu", "N") => 0.4
("Mn", "E") => 1.5
("Cu", "M") => 0.6
("Cu", "L") => 0.9
("C", "N")  => 0.0
("Mn", "F") => 1.3
("C", "M")  => 0.0
("C", "E")  => 1.5
("Mn", "L") => 0.0
=>
```

3(b): Implement, solve, and check the blend

Implement your LP in JuMP and solve with HiGHS. Print the status, cost, tons of each raw used, and total tons delivered. Report the amount of each raw left unused.

Compute the percentage of C, Cu, and Mn in the resulting blend from the solved quantities. Compare each with both its minimum and maximum requirement. Which quality limits hold with equality? Check all availability limits and recompute the cost from the quantities.

```
# Build and solve the blending LP here.
# Check mass, availability, each quality bound, and cost.

using JuMP, HiGHS

lose=Model(HiGHS.Optimizer)

@variable(lose, x[J] >=0)

@constraint(lose, available[j in J],
    0 <= sum(x[j]) <= u[j])

@constraint(lose, order,
    sum(x[j] for j in J) >= d)

@constraint(lose, makeup[e in E],
    α[e]sum(x[j] for j in J) <=
    sum(γ[e, j]x[j] for j in J))

@constraint(lose, makeup2[e in E],
    sum(γ[e, j]x[j] for j in J) <=
    β[e]sum(x[j] for j in J))

@objective(lose, Min, sum(c[j]x[j] for j in J))

optimize!(lose)
status = termination_status(lose)
status == MOI.OPTIMAL || error("HiGHS stopped with status $(status).")
is_solved_and_feasible(lose) || error("No feasible optimal solution is available")
println("Minimum Cost: ", objective_value(lose), " dollars")
solutions= Dict{(j) => value(x[j]) for j in J)
solution= Dict{(j) => value(x[j]) for j in J if value(x[j]) > 1e-6)

#Print Material Purchased\
println("")
for j in J
    if haskey(solution, (j))
        println(" Raw Mateial: ", j, " Amount Purchased: ", solution[j], " tons")
        println("Raw Material: ", j, " Amount Left Over: ", (u[j]-solution[j]),
```

```

else
    println("Raw Material: ", j, " Amount Left Over: ", (u[j]), " tons")
end
end

#Tons Delivered
delivered=sum(values(solution); init=0.0)
println("Amount of material delivered: ", delivered, " tons")

#Elements Used
println("")
for e in E
    element=sum(value(x[j])*γ[e,j] for j in J)/delivered
    println("Element: ", e, " Percentage: ", element)
end

#Print Cost
println("")
cost=sum(value(x[j])c[j] for j in J; init=0.0)
println("Cost calculated from outputs: ", cost, " dollars")

```

Running HiGHS 1.15.1 (git hash: 04024d701f): Copyright (c) 2026 under MIT license
Includes third-party software components, see THIRD_PARTY_NOTICES.md for full details
Using BLAS: blastrampoline
LP has 14 rows; 7 cols; 52 nonzeros
Coefficient ranges:
Matrix [1e-01, 3e+00]
Cost [2e+02, 2e+02]
Bound [0e+00, 0e+00]
RHS [2e+02, 6e+02]
Presolving model
6 rows, 7 cols, 39 nonzeros 0s
6 rows, 7 cols, 39 nonzeros 0s
Presolve reductions: rows 6(-8); columns 7(-0); nonzeros 39(-13)
Solving the presolved LP
Using dual simplex solver

Iteration	Objective	Infeasibilities	num(sum)
0	0.0000000000e+00	Pr: 1(250)	0.0s
3	9.0909090909e+04	Pr: 0(0)	0.0s

Performed postsolve
Solving the original LP from the solution after postsolve

Model status : Optimal
Simplex iterations: 3

Objective value : 9.0909090909e+04
P-D objective error : 0.0000000000e+00
HiGHS run time : 0.00
Minimum Cost: 90909.09090909091 dollars

Raw Material: F Amount Purchased: 45.45454545454541 tons
Raw Material: F Amount Left Over: 354.5454545454546 tons
Raw Material: U Amount Purchased: 136.3636363636364 tons
Raw Material: U Amount Left Over: 163.6363636363636 tons
Raw Material: E Amount Purchased: 318.18181818181824 tons
Raw Material: E Amount Left Over: 281.81818181818176 tons
Raw Material: L Amount Left Over: 500 tons
Raw Material: O Amount Left Over: 200 tons
Raw Material: N Amount Left Over: 300 tons
Raw Material: M Amount Left Over: 250 tons
Amount of material delivered: 500.0 tons

Element: C Percentage: 2.0000000000000004
Element: Cu Percentage: 0.40000000000000013
Element: Mn Percentage: 1.563636363636364

Cost calculated from outputs: 90909.09090909091 dollars

Results and interpretation for 3(b):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

The optimized solution is to purchase 45.45 tons F, 136.36 tons U and 318.18 tons E. This will result in a minimum cost of 90909.09 dollars delivering exactly 500 tons with a material makeup of 2% C, 0.4% Cu and 1.5636% Mn with the values of C and Cu being at the minimum quality limit. There will be 250 tons of M, 300 tons N, 200 tons O, 500 tons L, 281.81 tons E, 163.63 tons U, and 354.54 tons F left over from the supplier after this purchase.

3(c): At least 500 versus exactly 500

Without another solver run, show that **every** minimum-cost solution to this instance delivers exactly 500 tons.

If a feasible blend delivers $T > 500$ tons, consider multiplying every quantity by $500/T$. Explain what happens to composition, availability, delivery, and cost. Which assumption about the costs makes the improvement strict?

Would replacing the delivery constraint by equality leave the entire feasible set unchanged, or only preserve the optimal decisions and value for this instance? Explain.

Reasoning for 3(c):

Type your answer and reasoning in Markdown, or insert a clear photograph/scan of handwritten work. Label the subparts.

Every cost minimum solution delivers exactly 500 tons because any values that result in a tonnage greater than 500 can be scaled down to 500. This is because the feasible region is bounded on the lower end only by the 500 value and the composition requirements otherwise all the bounds are on the upper end (direction of maximizing). This means that if you were to change the inequality to an equality you would only preserve the optimal decisions and values for this instance and remove much of the feasible region.

Collaboration and LLM-use statement — required

Complete this statement as the **very last part of your homework**. Be specific enough that a reader can understand what work you did yourself and how others or LLMs helped you learn.

- **People:** Give the name of every person you collaborated with, identify the problem/subpart you discussed, and explain the nature of the collaboration. If you worked alone, explicitly state that you did not collaborate with anyone.
- **LLMs/tools:** Name every LLM or AI assistant you used and identify the model/version as precisely as available. If the interface did not show a model/version, say so. If you used none, explicitly state that you did not use an LLM or AI assistant.
- **Specific assistance and learning:** For each tool, identify the problem/subpart, describe what you asked it to help with, what suggestions you used or rejected, and how the interaction helped you understand the mathematics or implementation. Explain how you checked the suggestions. “I used ChatGPT for help” is not sufficiently specific.

Disclosure does not replace the requirement that your submitted formulation, implementation, and explanation be your own work and that you understand them.

People and collaboration:

I worked alone on this homework

LLMs/tools and models:

Google Gemini 3.5 Flash Lite

How I used the assistance, what I learned, and how I checked it:

I used this on problem 2 to assist in extracting values from a dictionary with tuples for the key. It provided the following format which `product_use = sum(i for ((product,c),i) in solution if product == p; init=0.0)` which I then had it explain and it provided :

`sum(...)`: The built-in Julia function that adds up a collection of numbers.

`for ((product, c), i) in solution:`

This iterates through your dictionary (assuming it's named `solution`).

It uses tuple destructuring, meaning it automatically unpacks each dictionary entry

if `product == p`:

A conditional filter. It tells the generator to only include values in the sum where

```
; init=0.0:
```

Sets the starting value of the sum to 0.0. This is a safety feature: if no match

I then used this information and formatting throughout the rest of the problem and extended some of what I learned to the following problem. I checked its results by asking for links to the documentation which it provided as https://docs.julialang.org/en/v1/base/collections/?utm_source=gemini#Base.sum, and <https://docs.julialang.org/en/v1/manual/arrays/#man-comprehensions>